

References

1. Contiki Home—<http://www.contiki-os.org/>
2. Talk on Contiki by Adam Dunkels—<http://webtv.compodium.se/trefpunkt22/007/>
3. Getting started with Instant Contiki—<http://www.contiki-os.org/start.html>
4. Develop your first app on Contiki—http://www.sics.se/contiki/wiki/index.php/Develop_your_first_application
5. Interconnecting Smart Objects with IP—The Next Internet [2010], Jean-Philippe Vasseur and Adam Dunkels

operate with public networks (such as Internet), more efficient and optimised protocols are also supported. One such example is the Rime stack, developed to optimise the IPv6 protocol.

Contiki comes with much documentation apart from well documented code. There are also community forums where active discussions happen.

Operating system functions include:

1. Process and memory management. Contiki supports memory block allocation and standard 'C' style allocation with `malloc()`. The concept and implementation of 'protothreads' has been introduced keeping the low system requirements in mind. 'protothreads' are C-language-based implementation minimising the overhead of multi-threaded programming.

2. Communication management. Contiki supports both IPv4 and IPv6 stack implementations, which include TCP, UDP and HTTP protocols with the smallest footprints. The IP stack is standards-compliant. The OS also supports low-power variants such as 6LoWPAN.

3. File system management. Not all devices as part of IoT have luxury of persistent storage such as Flash. But for such devices, Contiki provides support through the Coffee Flash file system.

Applications

There are several applications that come packaged as part of Contiki. These include a small Web browser, Web server, calculator, shell, telnet client and daemon, email client, vnc viewer and ftp.

Developers can find tools such as Cooja simulator for application development. Contiki also provides mechanisms to measure power for var-

ious operations that help developers produce applications that are power-sensitive. Software bundle also comes with a UNIX-style shell for interacting with OS and debugging.

Communication components in Contiki

uIP and uIPv6. There are several advantages of having the Internet protocol stack embedded in the OS. It allows communication with the existing devices/computers in the Internet. For this reason, the IP stack has been trimmed to fit 1 kB of RAM and a few kB of ROM, in comparison with Linux-based IP stack that requires 1MB RAM. This trimming of memory comes at the cost of reduced throughput.

uIPv6 is in contrast to the IPv6-certified version of IPv6, which was developed by Cisco. Support of IP stack along with TCP and UDP protocols makes it possible for devices running Contiki to directly interact with a Web server or tweak messages to the Internet.

LoWPAN (IEEE 802.15.4) and 6LoWPAN. Low-power wireless personal area networks have the characteristics of small packet sizes, low data rates, low-power devices and large number of devices. The IPv6 extension of the same is the 6LoWPAN standard developed by IETF working group. For saving power, the transmission and receiving periods are short spaced along with long periods of sleep mode by devices. Also, for power constraint reasons the communication is hop-based and short-range.

Getting started

To get started with Contiki, download the code and compile it on the target machine. Or, you can directly down-

load the virtual machine image, called Instant Contiki, to play around.

Below is the Hello World! program written by founding developer Adam Dunkels:

```
/*
 * \file
 * A very simple Contiki application
 * showing how Contiki programs look
 * \author
 * Adam Dunkels <adam@sics.se>
 */

#include "contiki.h"

#include <stdio.h> /* For printf() */
/*-----*/
PROCESS(hello_world_process, "Hello
world process");
AUTOSTART_PROCESSES(&hello_world_
process);
/*-----*/
PROCESS_THREAD(hello_world_process,
ev, data)
{
    PROCESS_BEGIN();

    printf("Hello, world\n");

    PROCESS_END();
}
/*-----*/
```

For compiling the code and developing applications, references are given in the box.

Similar software

TinyOS. It is an embedded operating system, developed by UC Berkeley in collaboration with Intel Corp, specifically designed for wireless sensor networks. Apart from internal architectural differences with Contiki, applications in TinyOS are developed using nesC, which is a dialect of 'C'.

FreeRTOS. It is a real-time OS designed with scalability and small footprint in mind. It supports preemptive and hybrid scheduling configurations. Supported devices include ARM, NXP, NEC, Infineon and Freescale. ●

The author is passionate about open source and productivity tools. He is currently a project manager at Hewlett-Packard